



What is Object-Oriented Programming, and How Does Encapsulation Work?

Object-oriented programming, also known as OOP, is a programming style in which developers treat everything in their code like a real-world object.

A class is like a blueprint for creating objects. Every single object created from a class has attributes that define data and methods that define the behaviors of the objects.

In a previous lesson, you learned how to create classes. Here's a reminder of the syntax:

```
class ClassName:
    def __init__(self, parameters):
        attribute = value

    def method_name(self):
        # method logic
```

Here's an example of a class that uses the `__init__` special method to initialize the brand and color attributes whenever an object is created using the class:

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

# create two objects from the Car class
car1 = Car('Toyota', 'red')
car2 = Car('Lambo', 'green')

print('Car 1 Brand:', car1.brand) # Car 1 Brand: Toyota
print('Car 1 Color:', car1.color) # Car 1 Color: red

print('Car 2 Brand:', car2.brand) # Car 2 Brand: Lambo
print('Car 2 Color:', car2.color) # Car 2 Color: green
```

Object-oriented programming has four key principles that help you organize and manage code effectively. They are encapsulation, inheritance, polymorphism, and abstraction.

The rest of this lesson will focus on how encapsulation works.

Encapsulation is the bundling of the attributes and methods of an object into a single unit, the class.

With encapsulation, you can hide the internal state of the object behind a simple set of public methods and attributes that act like doors. Behind those doors are private attributes and methods that control how the data changes and who can see it.

Let's say you want to track a wallet balance. You want to allow people to deposit or withdraw money from the wallet, but no one should be able to tamper with the balance directly.

In that case, you can make `deposit()` and `withdraw()` public methods, and you hide the balance under the `_balance` attribute:



```
self._balance = balance # For internal use by convention
```

```
def deposit(self, amount):
    if amount > 0:
        self._balance += amount # Add to the balance safely

def withdraw(self, amount):
    if 0 < amount <= self._balance:
        self._balance -= amount # Remove from the balance safely
```

By convention, prefixing attribute and methods with a single underscore means they are meant for internal use. No one should directly access them from outside the class since it defies the principles of encapsulation, which can lead to bugs.

While a single underscore prefix is just a convention, prefixing attributes and methods with a double underscore effectively prevents them to be accessed from the outside of their class, making those attributes and methods private.

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount # Add to the balance safely

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount # Remove from the balance safely

account = Wallet(500)
print(account.__balance) # AttributeError: 'Wallet' object has no attribute '__balance'
```

To get the current value of `__balance`, you can define a `get_balance` method. For example:

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount

    def get_balance(self):
        return self.__balance

acct_one = Wallet(100)
acct_one.deposit(50)
```



```
acct_two.withdraw(28)
print(acct_two.get_balance()) # 422

acct_two.deposit(150)
print(acct_two.get_balance()) # 572
```

You can also define a private `__validate` method to check if every deposit or withdrawal amount is a positive number:

```
class Wallet:
    def __init__(self):
        self.__balance = 0

    def __validate(self, amount):
        if amount < 0:
            raise ValueError('Amount must be positive')

    def deposit(self, amount):
        self.__validate(amount)
        self.__balance += amount

    def withdraw(self, amount):
        self.__validate(amount)
        if amount > self.__balance:
            raise ValueError('Insufficient funds')
        self.__balance -= amount

    def get_balance(self):
        return self.__balance

acct_one = Wallet()
acct_one.deposit(4) # ValueError('Amount must be positive')
print(acct_one.get_balance()) # 0

acct_one.deposit(50)
print(acct_one.get_balance()) # 50
acct_one.withdraw(-8) # ValueError('Amount must be positive')
acct_one.withdraw(58) # ValueError('Insufficient funds')
```

As you can see, the `__validate` method is private, and runs behind the scenes in the `deposit()` and `withdraw()` public methods to make sure the amount is always valid.

In a coming lesson, you will learn more about how attributes prefixed with a double underscore works.

In summary, encapsulation locks down internal data behind clear public methods. That's how you keep your classes safe from tampering and centralize validation in one place. You can update or extend your code freely, knowing that outside code only touches the interfaces you expose.

Questions

What are the four key principles of Object-Oriented Programming (OOP)?



Python Certification

Understanding Object Oriented Programming and Encapsulation

Correct!

Variables, Functions, Loops, Conditionals

Classes, Objects, Methods, Attributes

Public, Private, Protected, Static

Why is it a bad practice to directly access attribute/methods prefixed with a single underscore?

Because Python will raise a syntax error if you try.

Because it violates encapsulation, allowing uncontrolled data access.

Correct!

Because private properties are slower to access than public ones.

Because they can only be accessed from child classes.

What is the way for indicating private attributes or methods?

Prefix with a single underscore

Use leading and trailing double underscores

Prefix with a double underscore

Correct!

Use the private keyword

Check your answer

Ask for Help



Python Certification



Understanding Object Oriented Programming and Encapsulation